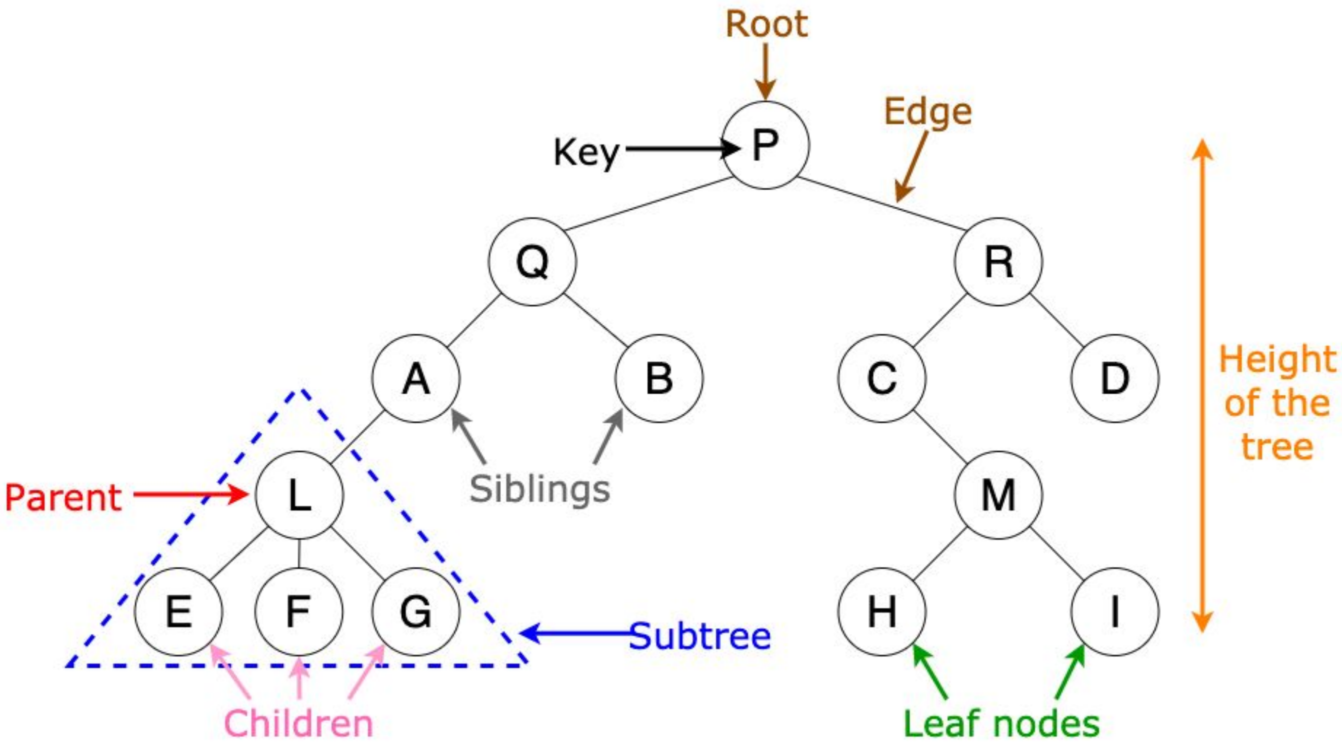
Data Structures

Tree (Basics)

Slides taken from Prantik Paul [PKP]

Trees


Root
Key
P
Edge
Q
R
A
B
C
D
Height of the tree
L
M
Parent
Siblings
E
F
G
H
I
Subtree
Children
Leaf nodes

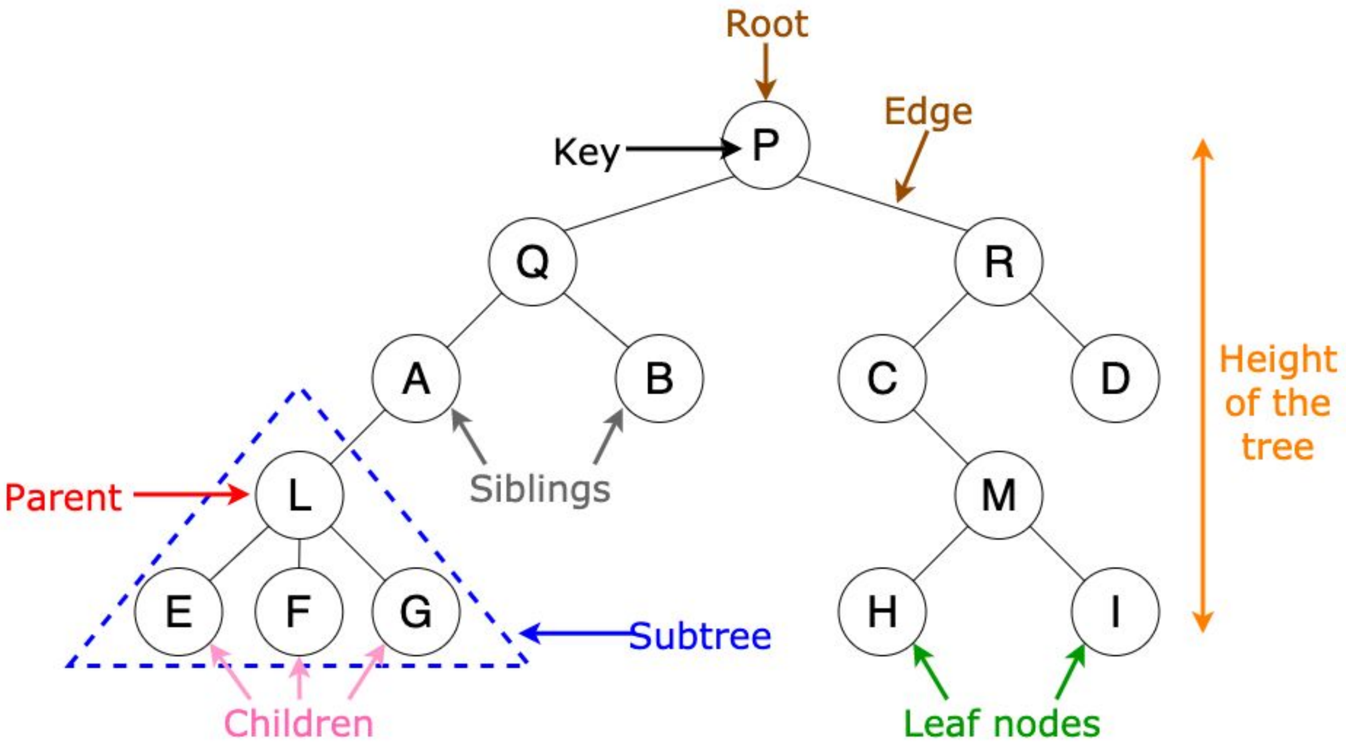
Why Trees?

Sorting New Elements

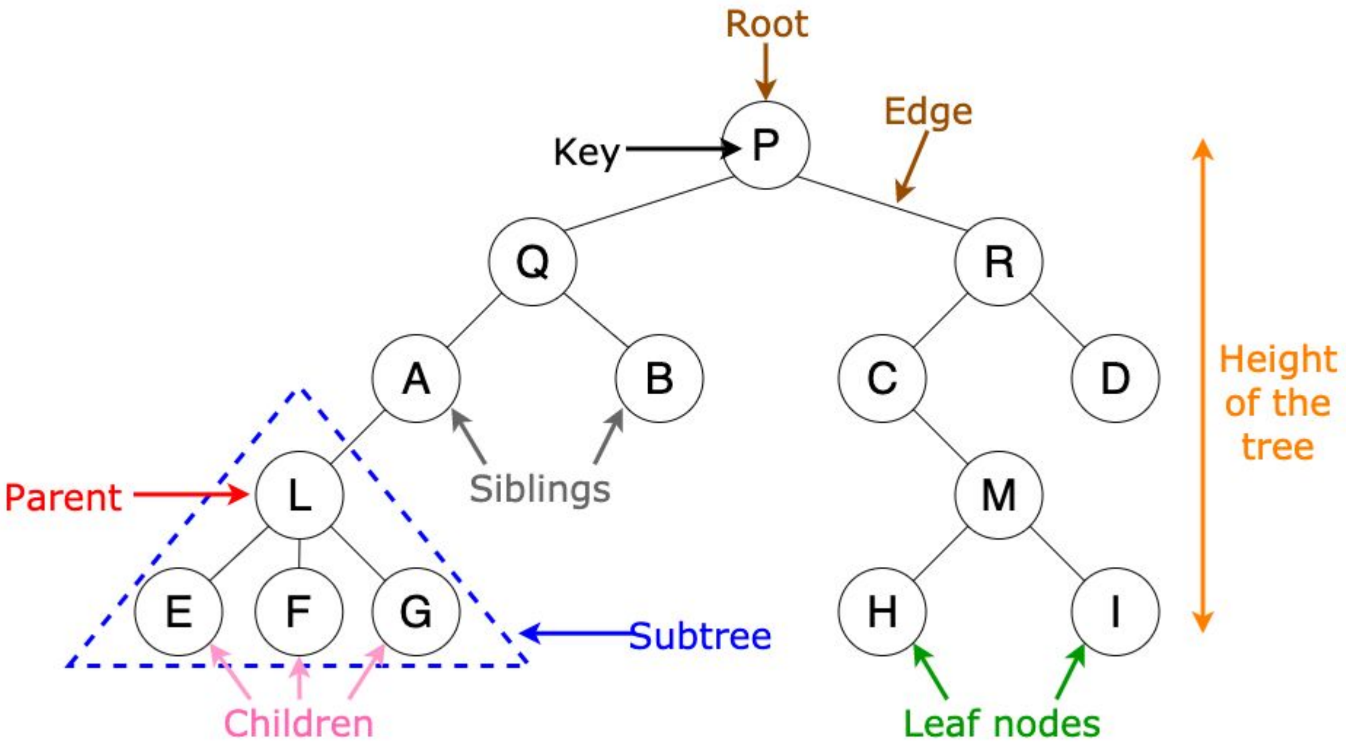
Folder/File System Structure

Computer Network Algorithms

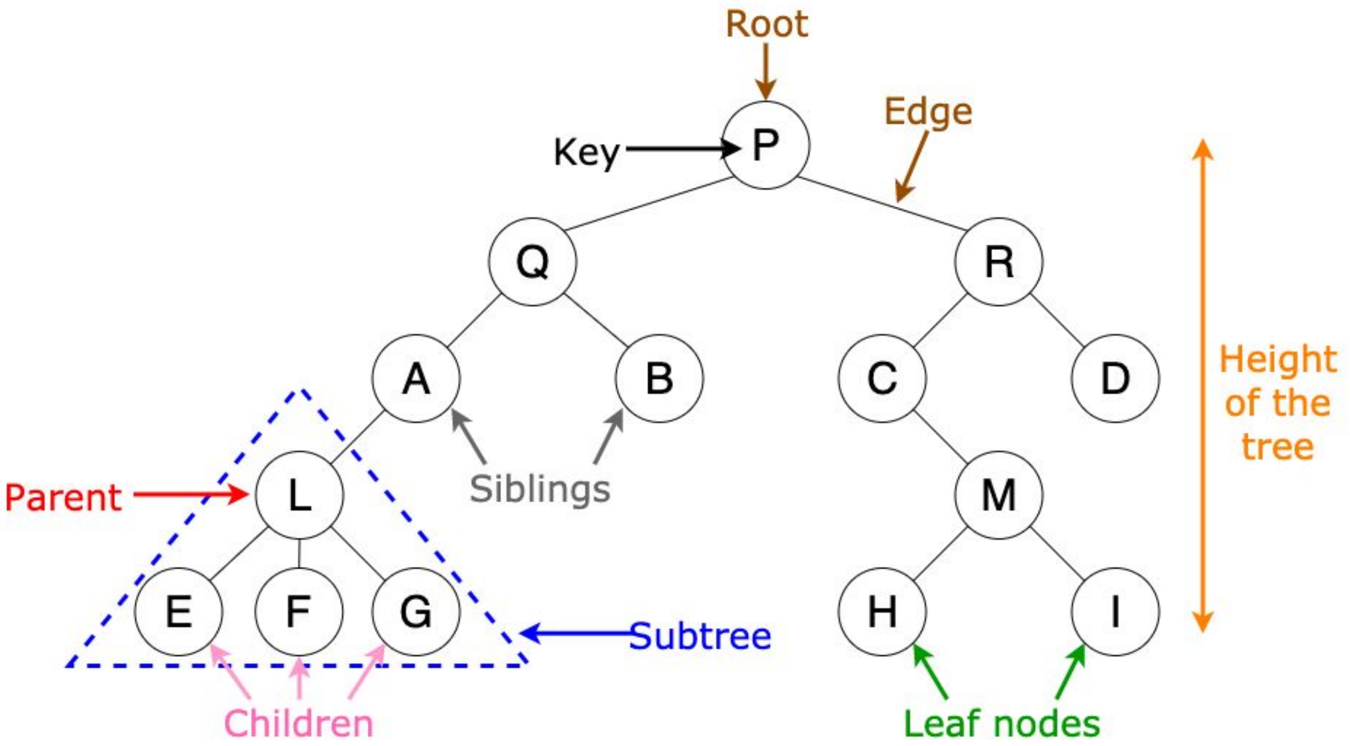
Trees - Root/Leaf/Non-Leaf



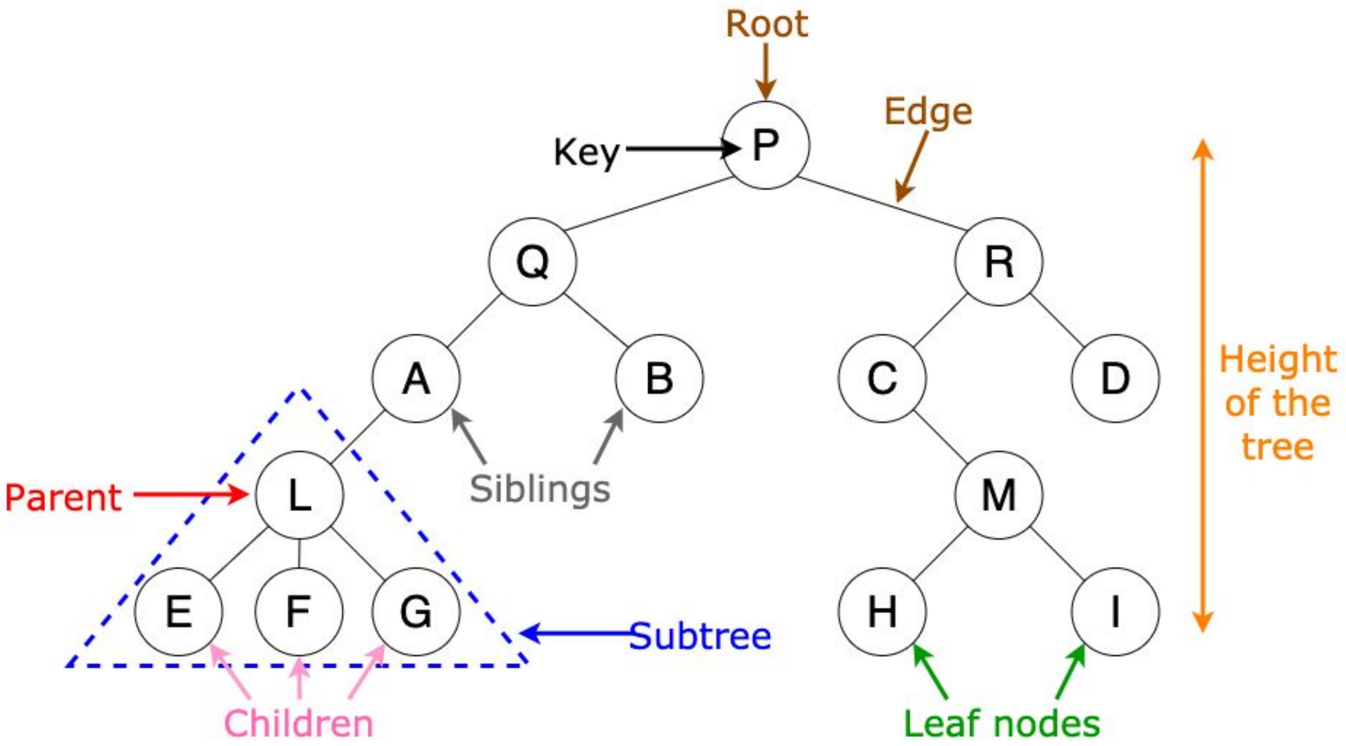
Trees - Parent/Child/Siblings



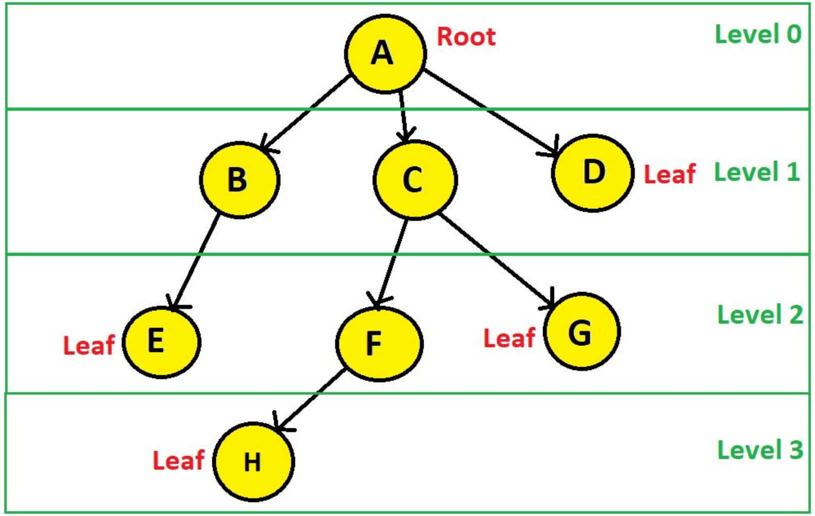
Trees - Edge/Path



Trees - Degree



Trees - Depth/Height/Level

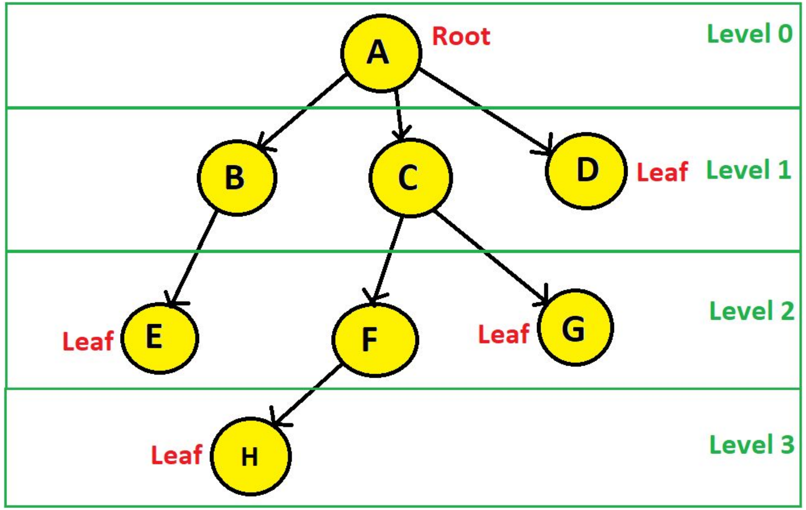


Depth of a node: The level of a node in the tree, with the root having depth 0 and each level of children increasing the depth by 1.

Level of a node == Depth of that node.

Height of a node: The length of the longest path from a node to a leaf. The height of the tree is the height of the root node.

Trees - Depth/Height/Level



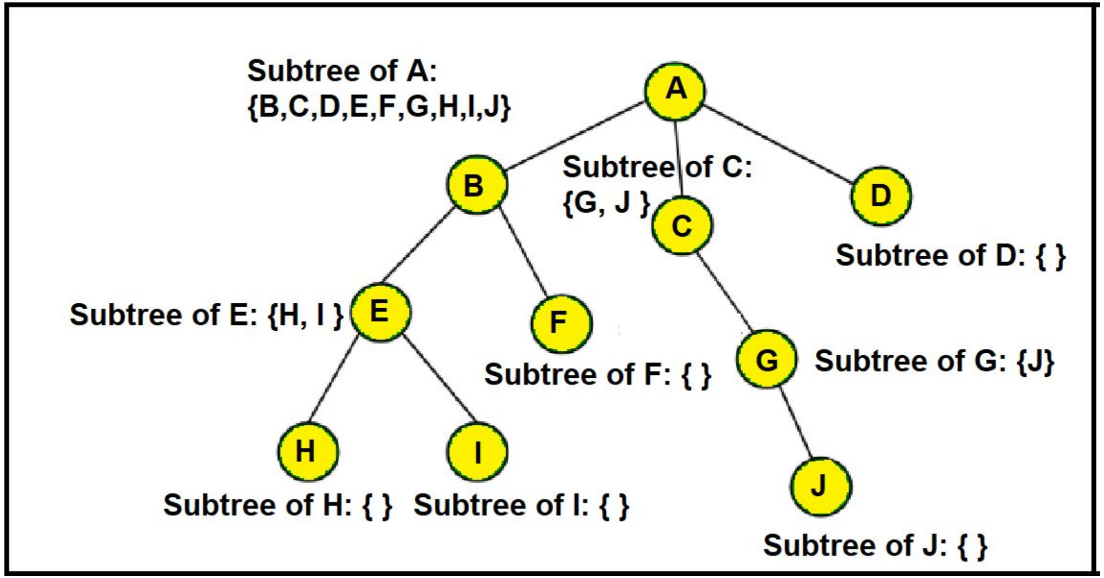
Depth of A?

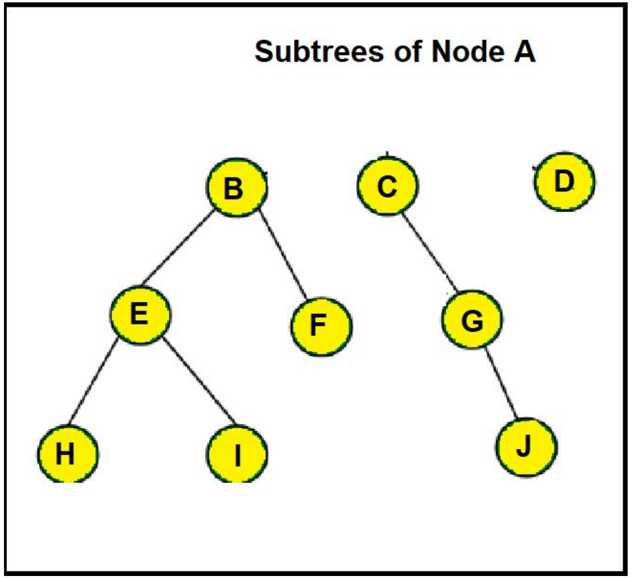
Height of A?

Depth != Height

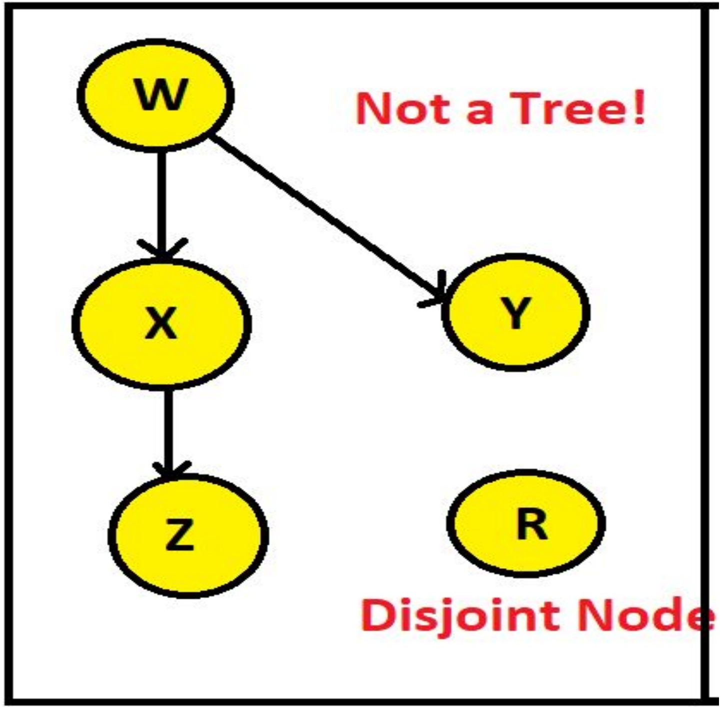
Depth = Level

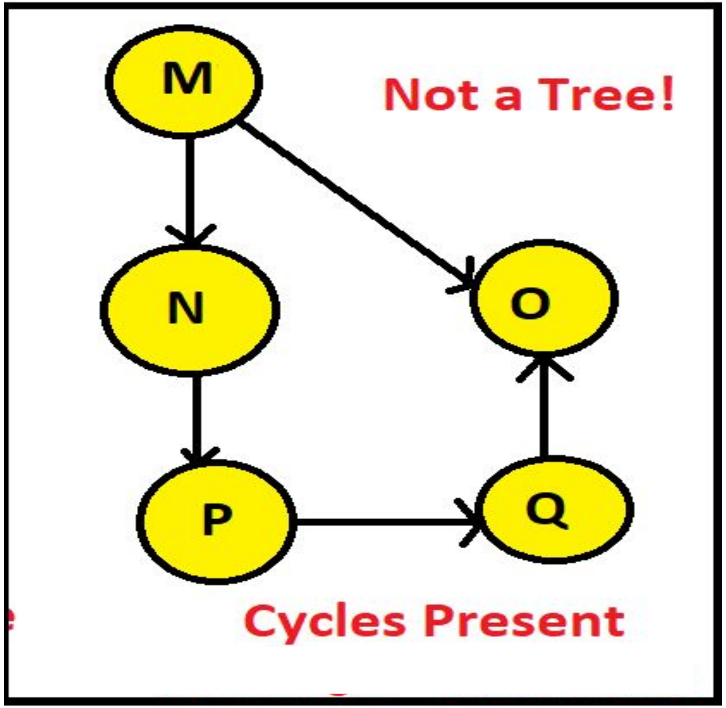
Trees - Subtree



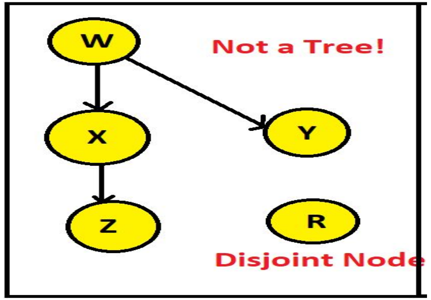


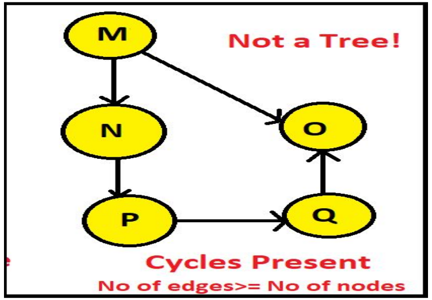
Trees - Characteristics





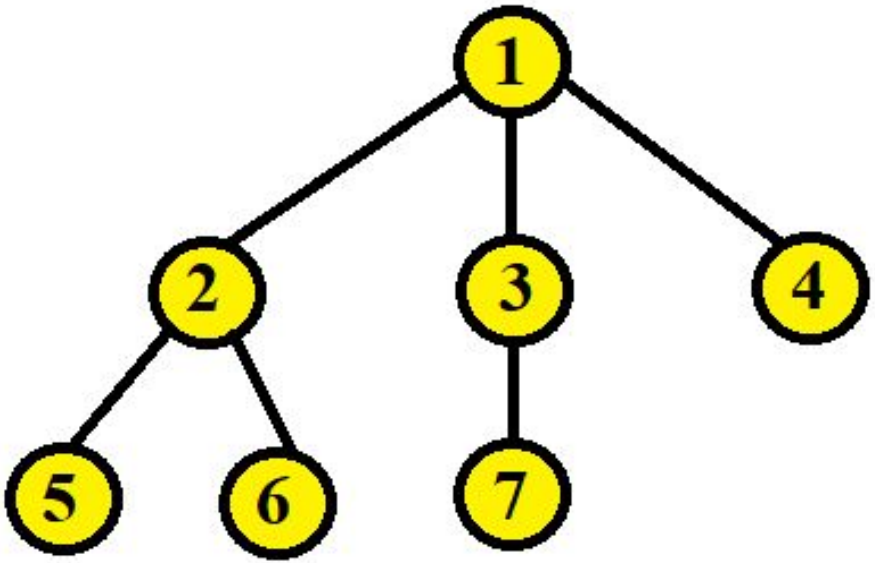
Trees - Characteristics





- A tree must be continuous and not disjoint, which means every single node must be traversable starting from the root node.
- A tree cannot have a cycle. A tree having n number of nodes will have n or more edges if it contains one or more cycles. Which means, for a tree, no of edges == no of nodes - 1.

Trees - Build a Tree



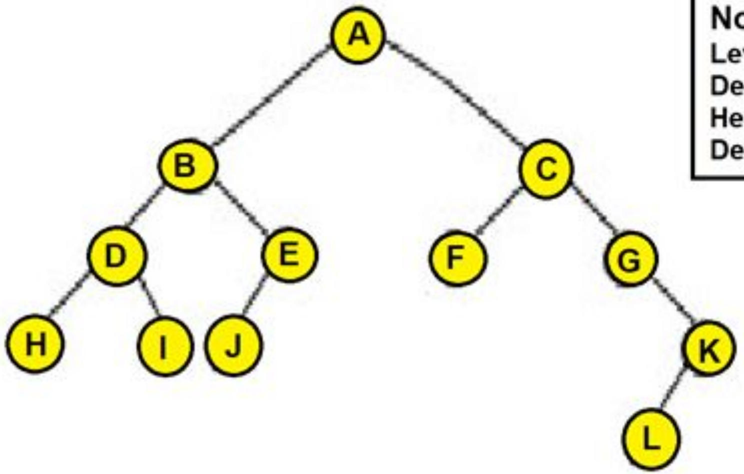

```python
class Node:
    def __init__(self, elem):
        self.elem = elem
        self.children = []

root = Node(1)
root.children += [Node(2)]
root.children += [Node(3)]
root.children += [Node(4)]
root.children[0].children += [Node(5)]
root.children[0].children += [Node(6)]
root.children[1].children += [Node(7)]
```

Dynamic Representation with Linked List

Sequential Representation with Array

Practice Problem 1



Node A
Level:
Depth:
Height:
Degree:

Node D
Level:
Depth:
Height:
Degree:

Node F
Level:
Depth:
Height:
Degree:

Node C
Level:
Depth:
Height:
Degree:

Node E
Level:
Depth:
Height:
Degree:

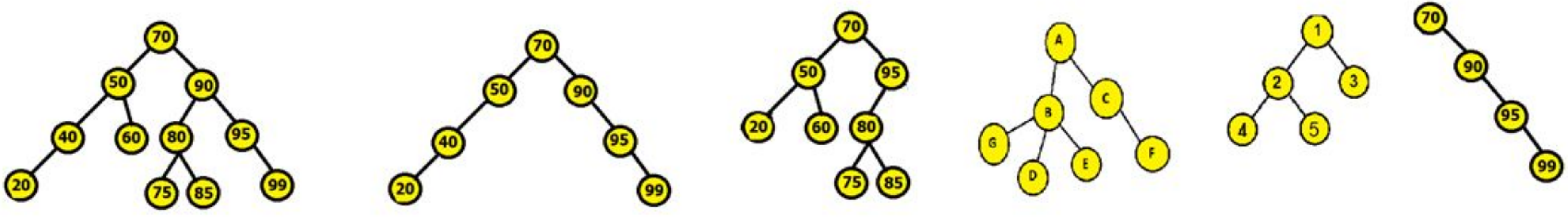
Node G
Level:
Depth:
Height:
Degree:

Node L
Level:
Depth:
Height:
Degree:

Find the level, depth, height and degree of the specified nodes of the following tree.

Practice Problem 2

Identify which of the following trees are full, complete, perfect and balanced.



Will learn later !

Practice Problem 3

Write the code to construct a tree of height 3 and minimum number of 9 nodes. Use your imagination while designing the tree.